pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

The ML Workflow and Simple Regression

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Course 1's line becomes a model

Course 1, notebook 12

OAS on rating score, 200 holdings bonds: 42.45 bp per notch, R squared 0.55. Drawn through every bond; nothing held back.

At score 1: -169.17 bp

Fit

on the training rows only

Judge

on test rows it never saw

Measure

errors in bp

821 bonds of 150 invented issuers. Their spreads come from the course's own script: rating, maturity, an issuer effect, noise.

Synthetic data: a regression here recovers part of a recipe the course wrote, and nothing about a real market.

Why the seed is 42

scikit-learn's glossary, random_state

"... it may be worthwhile checking that your results are stable across a number of different distinct random seeds. Popular integer random seeds are 0 and 42." The 42 links to an article on The Hitchhiker's Guide to the Galaxy.

Douglas Adams, 1993, when asked why 42

"The answer to this is very simple. It was a joke. It had to be a number, an ordinary, smallish number, and I chose that one."

Any whole number makes a run repeatable. No number makes a model better.

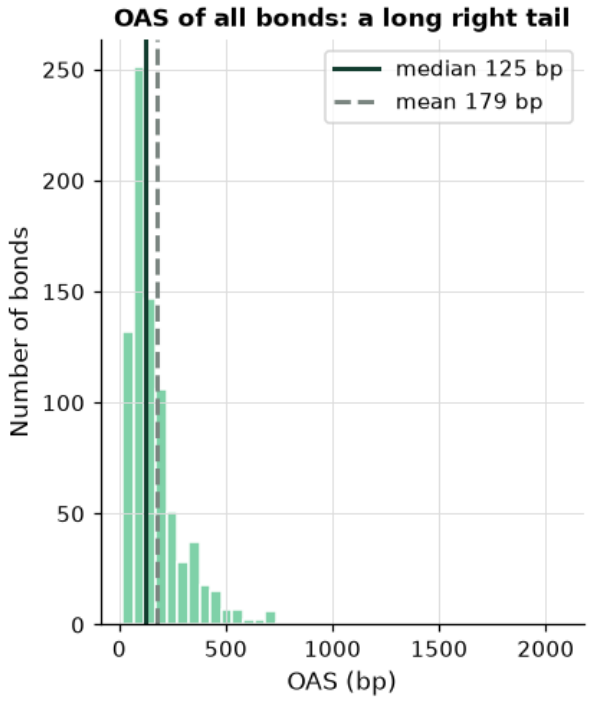
Sources, read 4 October 2026: scikit-learn glossary, random_state (scikit-learn.org); Wikipedia, Phrases from The Hitchhiker's Guide to the Galaxy (secondary); D. Adams, alt.fan.douglas-adams, 3 Nov 1993, Internet Archive copy.

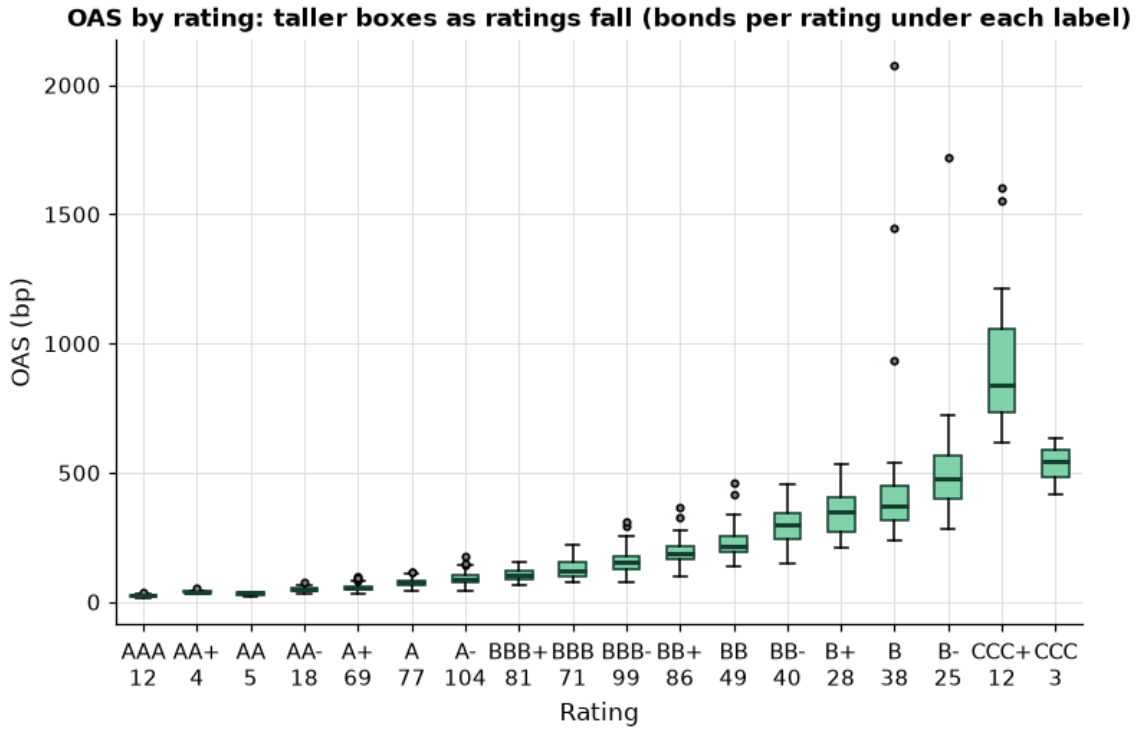
What machine learning is here

A function fitted to rows of data, then judged on rows it did not see.

Word	Meaning	Today
Row	One observation	One bond
Feature	A column the function reads	Rating score, 1 (AAA) to 18 (CCC)
Target	The column it is fitted to	OAS in bp
Model	A function with fitted numbers	A straight line: intercept and slope
Training rows	The rows the numbers are fitted on	615 bonds
Test rows	Kept out; used once to report	206 bonds

Look at the bonds before fitting anything





The mean (179 bp) sits above the median (125 bp): a few very wide spreads. The boxes grow as ratings fall.

The workflow, and where it fails in finance

Step	Today
1. Question	OAS and rating score, in this file
2. Rows and columns	821 bonds; score; oas in bp
3. Split first	615 training, 206 test
4. Fit	A straight line, training rows
5. Choose	Nothing today (week 2)
6. Report	Test rows, once: errors in bp
7. Describe	What it shows, where it fails

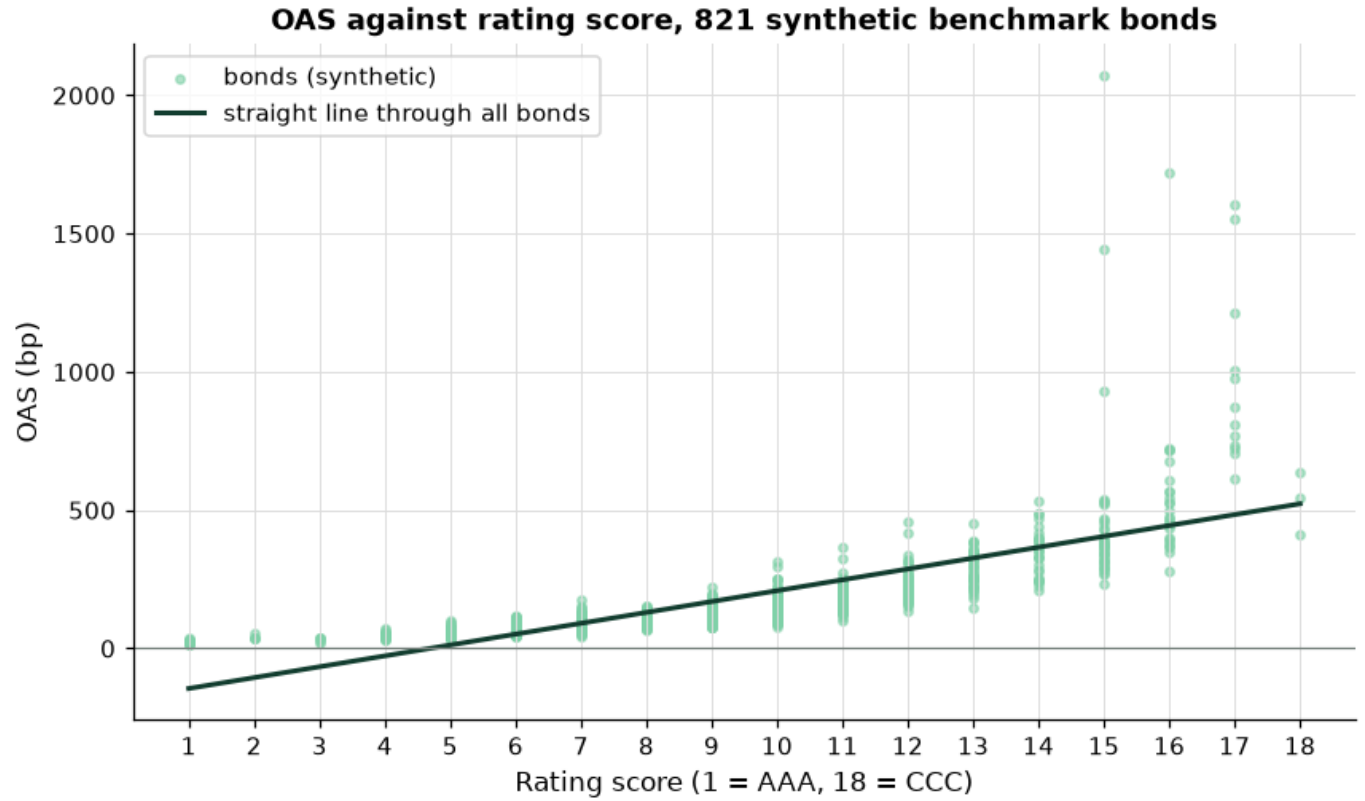
Failure	Looks like
Lookahead	A date's number uses later data
Leakage	Test rows inside the training
A metric that hides	Right on most, no defaults caught
Rows not independent	Bonds of one issuer
A model read as a view	A fitted spread as a verdict

Each one flatters the model and none raises an error. Today: 150 issuers, 3 to 8 bonds each, 5.5 on average.

How a least squares line works

The miss	Actual OAS minus the line's value at the bond's score, in bp
Squared and summed	Squaring stops a bond above and a bond below from cancelling
The winner	The one line with the smallest sum of squared misses; np.polyfit finds it in one calculation
You control	The feature, and the rows the line is drawn through
What comes out	A slope in bp per notch and an intercept in bp

Course 1's line, on all 821 bonds



39.2998

bp per notch

0.5289

R-squared

-145.1 bp at AAA

A straight line; the spreads curve.

Excel, all bonds (tested)

```
=SLOPE(oas, score)
=INTERCEPT(oas, score)
=RSQ(oas, score)
```

How the train and test split works

The problem	A model always looks good on the rows it was fitted to
Shuffle and cut	train_test_split shuffles the row labels with a seed, then cuts them in two
You control	test_size, the share put away, and random_state, which fixes the shuffle
What comes out	Features and target for each side, with the bonds' labels kept
The rule	Split before any fitting, scaling, or choosing

The split: put the test rows away first

615 training rows: fit | **206 test rows**

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=SEED)
```

mlkit.assert_disjoint

Stops if a label is on both sides. Five labels copied on purpose: "5 row labels are in both the training and the test rows"

Issuers on both sides

203 of 206 test bonds share an issuer with a training bond. Fine for score alone; day 5 splits by issuer.

The test rows are used once, for the reported numbers.

How LinearRegression works

Build	An empty model: no numbers yet
Fit	Finds the intercept and one slope per feature that make the squared misses smallest on the training rows
Keeps	Only coef_ and intercept_; it never looks at the training rows again
Ask for output	predict(X) multiplies each row by the slopes and adds the intercept, for any bond
What comes out	Slopes in bp per unit of each feature, and a fitted value in bp per row

scikit-learn: build, fit, then ask for output

1. Build

```
model = LinearRegression()
```

2. Fit

```
model.fit(X_train, y_train)
```

3. Output

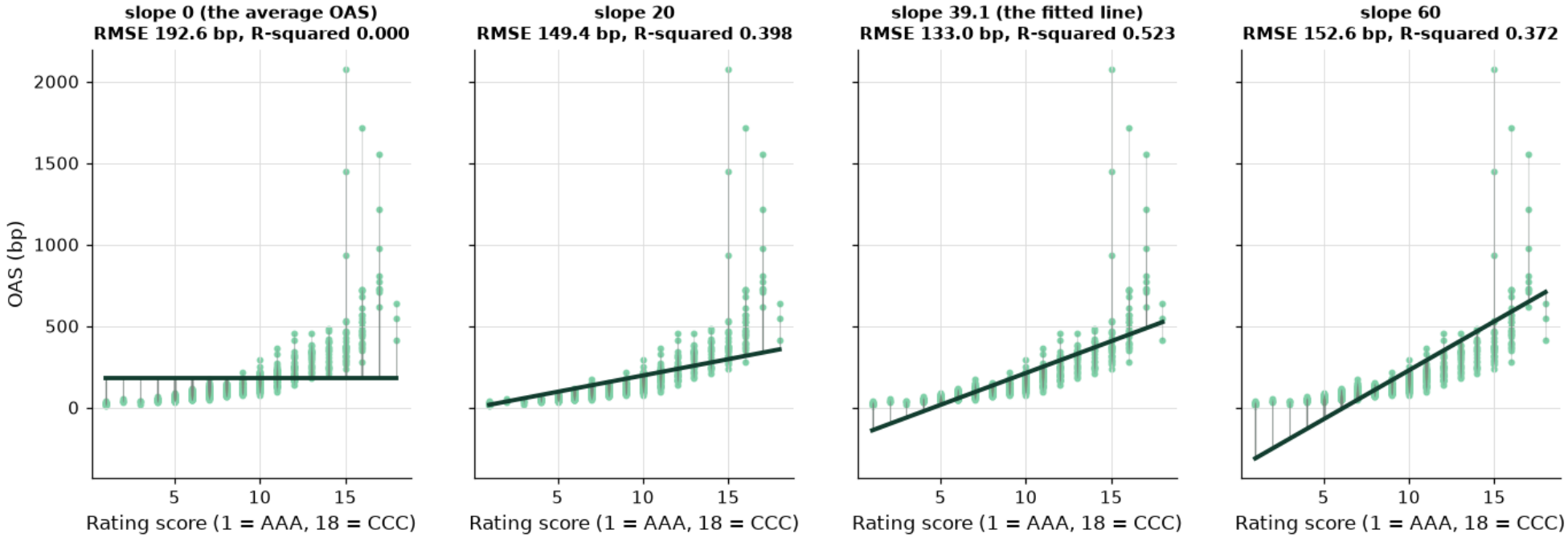
```
model.predict(new_rows)
```

predict is the method's name: the fitted value for the rows given. It never refers to the future of a market.

Rating	Fitted spread (bp)
AAA	-140.0
A	55.3
BBB	172.4
BB	289.6
B	406.7
CCC	523.9

Slope 39.0526 bp per notch, intercept -179.0571 bp, from 615 training rows.

Least squares, by eye



RMSE in bp: slope 0 192.6, slope 20 149.4, fitted slope 39.1 133.0, slope 60 152.6. The fitted line is the bottom of the curve.

How the three error measures work

The miss	Actual OAS minus the fitted value, in bp, for every bond
MAE	Averages the sizes of the misses, ignoring sign: the typical miss in bp
RMSE	Squares, averages, then takes the root: in bp, and large misses count more
R-squared	Compared with a flat line at the average: 1 is no miss, 0 is no better, below 0 is worse
What comes out	Three numbers per set of rows; read the test MAE first

Three error measures, in bp

R-squared

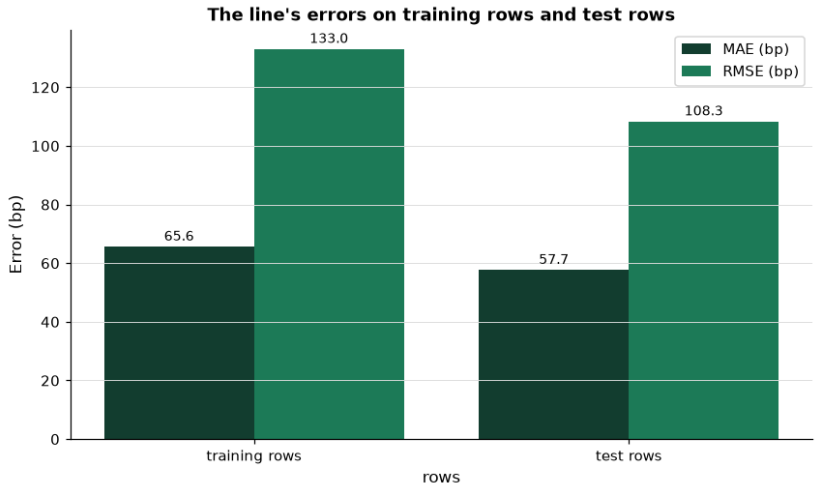
Share of variation accounted for; can be negative on test rows

MAE (bp)

The typical miss: mean absolute error

RMSE (bp)

Root mean squared error: large misses count more

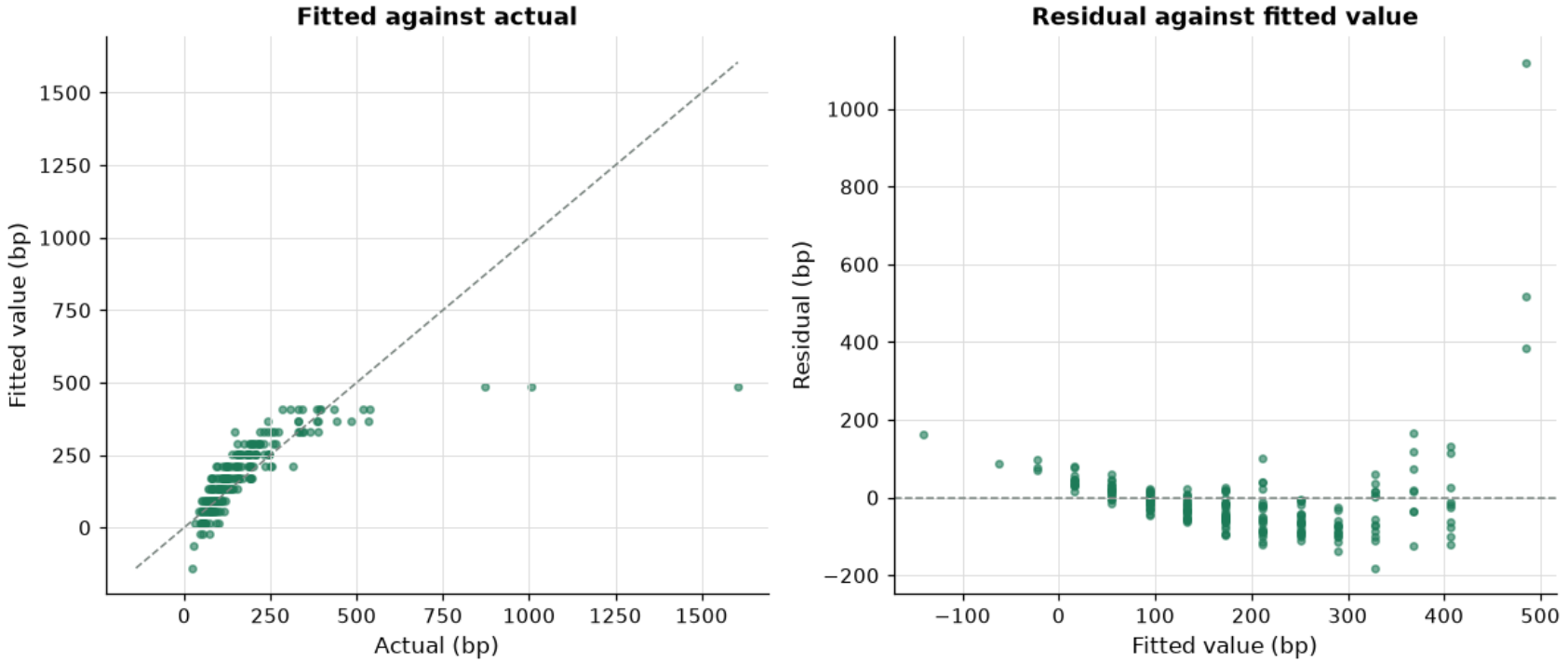


	R-squared	MAE (bp)	RMSE (bp)
Training rows	0.5227	65.6	133.0
Test rows	0.5529	57.7	108.3

5 passed regression_metrics and assert_disjoint, tested

The main number: test MAE in bp. Test better than training is the luck of one split.

The line's misses, on a chart



A good model leaves a band of residuals the same width all the way across. This band widens: the wrong shape.

What the fitted line is, and what it is not

Allowed names

fitted spread

the model's spread for these characteristics

residual: actual minus fitted, in bp

Never called

fair value, fair spread

cheap, rich, mispriced

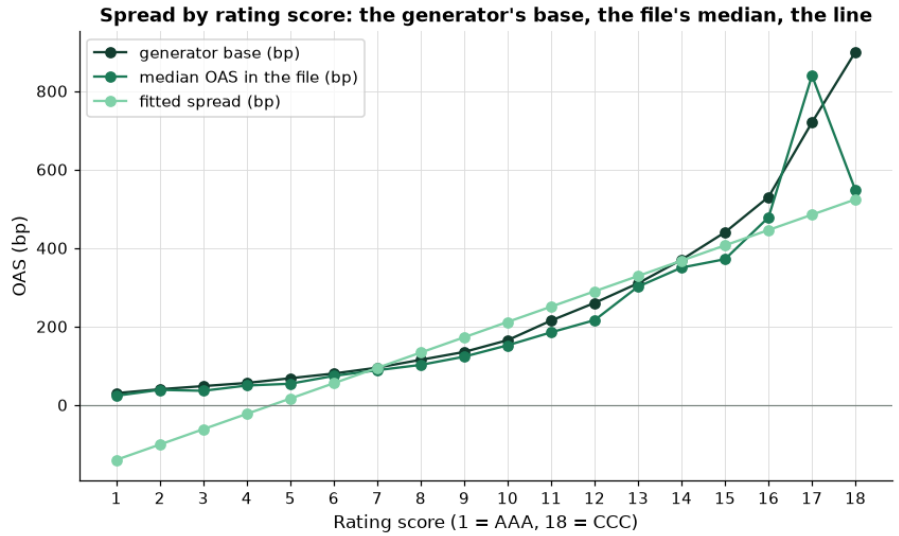
a signal, a target, a trade

The line knows one column of invented bonds. 5 test bonds get a fitted spread below zero: a sign of the wrong shape, not a finding about them.

A residual says how much of a bond's spread the line misses. Nothing about what anyone should do.

Where these spreads came from

```
oas = round( base(score) x (0.70 + 0.30 x min(T, 10) / 10) x issuer effect x noise )
```



A regression can recover

In logs the recipe adds: the rating table, closely (day 4).

It cannot recover

The issuer effect without an issuer feature; the noise; which six bonds were made distressed.

Sector plays no part. Every number fitted here describes a recipe the course wrote.

Excel side by side

Excel side by side (tested in Excel)

`=SLOPE(A2:A616,B2:B616)  (train)`	slope 39.0526 bp per notch
`=RSQ(A2:A616,B2:B616)  (train)`	R-squared 0.5227
`=train!$E$2+train!$E$1*B2  (test, C2 down)`	the fitted spread
`=AVERAGE(ABS(A2:A207-C2:C207))`	MAE 57.7 bp
`=SQRT(SUMXMY2(A2:A207,C2:C207)/COUNT(A2:A207))`	RMSE 108.3 bp
`=1-SUMXMY2(A2:A207,C2:C207)/DEVSQ(A2:A207)`	test R-squared 0.5529
`=RSQ(A2:A207,C2:C207)`	0.5592: not the test R-squared

Entered the old way, the MAE formula is stored with @ and keeps one row: 42.8 bp in row 6, #VALUE! below the data.

Excel has no split function: the rows are split in Python and pasted onto two sheets.

Save the day: the experiment log

1. Init

The work folder, local settings, the guard

2. .gitignore

__pycache__/, .pytest_cache/, *.xlsx

3. Code

mlkit.py and test_mlkit.py

4. Results

results.csv, 4 decimals

```
model,features,train_rows,test_rows,test_r2,test_mae_bp,test_rmse_bp
oas_on_score_line,score,615,206,0.5529,57.6620,108.3312
```

A model's results are committed beside its code, so a change is reviewed by its numbers too.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the line is fitted on all 821 bonds, then scored on the test rows: 56.9 bp against 57.7 bp.

Code that runs is not the same as code that is right.

What you can do now

- A function fitted to rows, judged on rows it did not see
- Split first; prove it with assert_disjoint
- Build, fit, then ask for the output
- Errors in bp: the test MAE is the main number
- A fitted spread is a fitted spread, nothing more

Week 1, Day 2 Multiple regression and dummy variables: a step for every rating